

I grafi

Si dice grafo un insieme G di coppie (E,V) costituito da:

- un insieme V di **nodi** (o **vertici**)
- un insieme E di **archi** (o **spigoli**)

Vi sono due tipi di grafi

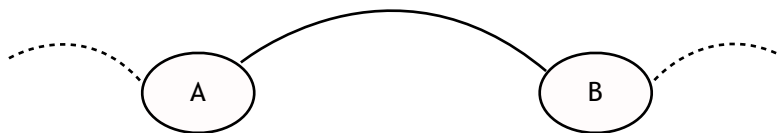
- Grafi **non orientati** (gli archi non hanno un verso - coppie non ordinate)
- Grafi **orientati** (gli archi hanno verso - coppie ordinate)

Multi-archi: archi distinti aventi gli stessi estremi

Cappio: arco con estremi coincidenti

Un grafo dove sono permessi multi-archi e cappi è detto **multi-grafo**.

Grafi non orientati



L'arco $\{A,B\}$ è **incidente** sui nodi A e B;

I nodi A e B sono **adiacenti** poiché A è adiacente a B e B è adiacente ad A.

Il **grado** di un nodo è la quantità di archi incidenti sul nodo e si indica con delta δ .

In grafi orientati, il grado di un nodo è la quantità di archi **uscenti** sommata alla quantità degli archi **entranti**.

Altre proprietà

Dato un grafo G , indichiamo con n il **numero di nodi** e con m il **numero di archi**.

- La somma dei gradi dei nodi di un grafo è il doppio del numero di archi ($2m$) (ogni arco è sia entrante da un nodo che uscente da un'altro nodo, quindi "conta due").
- In un grafo orientato la somma dei gradi uscenti, uguale alla somma dei gradi entranti, è uguale al numero di archi m .

In un grafo non orientato vi possono essere al più tanti archi quante sono le coppie non ordinate distinte di nodi. Quindi, dati n nodi vi sono al più:

$$n + (n-1) + (n-2) + \dots = n(n+1)/2 = O(n^2) \text{ archi.}$$

Grafi Completi

Un grafo che ha un arco tra ogni coppia di vertici distinti è detto **grafo completo**.

Solitamente non si hanno mai grafi completi quindi la complessità n^2 è rara da trovarsi, ma comunque può accadere. Un grafo il cui numero di archi sia dell'ordine di n^2 è detto **grafo denso**. Altrimenti è detto **grafo sparso**.

Il cammino di un grafo

Un cammino è una serie di vertici ed archi che si susseguono. La lunghezza del cammino è il numero totale di archi che collegano i vertici della sequenza.

- **cammino nullo**: cammino di lunghezza 0.
- **cammino non nullo**: detto anche cammino proprio, è un cammino di lunghezza maggiore di zero.
- **cammino semplice**: un cammino è detto semplice se tutti i suoi vertici sono distinti (compaiono una sola volta nella sequenza).
- **cammino chiuso**: un cammino (proprio) è detto chiuso se l'ultimo vertice del cammino coincide col primo.

I cicli

- **ciclo semplice**: è un cammino sempre chiuso che non attraversa nessun nodo più di una volta. Se non sono permessi i cappi un ciclo di un grafo orientato ha lunghezza maggiore o uguale a 2.

Le complessità degli algoritmi

Algoritmi di ordinamento (basati sul confronto)

Selection sort:

Complessità temporale: $T(n) = \Theta(n^2)$

Complessità spaziale: $S(n) = \Theta(1)$

In place: sì.

Stabile: no.

Insertion sort:

Complessità temporale:

- Best: $T(n) = \Theta(n)$

- Middle: $T(n) = \Theta(n^2)$

- Worst: $T(n) = \Theta(n^2)$

Complessità spaziale: $S(n) = \Theta(1)$

In place: sì.

Stabile: sì.

Mergesort:

Complessità temporale: $T(n) = \Theta(n \cdot \log(n))$

Complessità spaziale: $S(n) = \Theta(n + \log(n))$

In place: no.

Stabile: sì.

Quicksort:

Complessità temporale:

- Best: $T(n) = \Theta(n \cdot \log(n))$

- Middle: $T(n) = \Theta(n \cdot \log(n))$

- Worst: $T(n) = \Theta(n^2)$

Complessità spaziale:

- Best: $T(n) = \Theta(n \cdot \log(n))$

- Middle: $T(n) = \Theta(n \cdot \log(n))$

- Worst: $T(n) = \Theta(n)$

In place: no.

Stabile: no.

Heapsort:

Complessità temporale: $\Theta(n \cdot \log(n))$

Complessità spaziale: $\Theta(1)$

In place: sì.

Stabile: no.

Algoritmi di ordinamento (non basati sul confronto)

Couting Sort:

Complessità temporale: $T(n, k) = \Theta(n + k)$

Complessità spaziale: $S(n, k) = \Theta(n + k)$

In place: sì.

Stabile: sì.

Backle Sort:

Complessità temporale: $T(n, k) = \Theta(n + k)$

Complessità spaziale: $S(n, k) = \Theta(n + k)$

In place: sì.

Stabile: sì.

Radix Sort:

Complessità temporale: $T(n, k) = \Theta(n + k)$

Complessità spaziale: $S(n, k) = \Theta(n + k)$

In place: sì.

Stabile: sì.

$T(n, k) = \text{bitTotaliElemento} / r \cdot (n + r^2)$

Si sceglie un r che sia tendenzialmente $\log(n)$.

Le complessità delle strutture dati

Array non ordinato:

Add: $O(1)$

Delete: $O(n)$

Find: $O(n)$

Array ordinato:

Add: $O(n)$

Delete: $O(n)$

Find: $O(\log(n))$ - ricerca binaria

Lista non ordinata:

Add: $O(1)$

Delete: $O(n)$

Find: $O(n)$

Lista ordinata:

Add: $O(n)$

Delete: $O(n)$

Find: $O(n)$

Alberi AVL (unobilanciati):

Add: $\Theta(\log(n))$

Delete: $\Theta(\log(n))$

Find: $\Theta(\log(n))$

Coda con priorità (heap):

Move Up: $\Theta(\log(n))$

Move Down: $\Theta(\log(n))$

Add: $\Theta(\log(n))$

Delete: $\Theta(\log(n))$

Find: $\Theta(\log(n))$

Eject First: $\Theta(\log(n))$ (move Down)

Change Priority: $\Theta(\log(n))$ (move Up/Down)

Heapify: $\Theta(n * \log(n))$

Union Find:

(Find) Path Compression: $\Theta(1)$ (ammortizzato)

Find (con albero equilibrato): $O(\log(n))$

Find (con albero degenerare): $O(n)$

Union: $\Theta(1)$

Grafi con liste di adiacenza:

Dijkstra (cammini minimi): $\Theta(n + m)$

Kruskal (minimo ricoprente): $\Theta(n + m)$

Prim (minimo ricoprente): $\Theta(n + m)$

Componenti Fortemente Connesse: $\Theta(n + m)$

Ordine topologico: $\Theta(n + m)$

Notazioni:

- *inizio_visita(u)* e *fine_visita(u)* rappresentano i momenti in cui è possibile agire sul nodo. *visita(u)* è usato invece nei metodi iterativi che hanno un solo punto di “azione”.
- *padri* è un array che contiene l'albero risultato (lettura, cammino minimo, etc) indicizzato sui nodi del grafo. Quando *padri[u] = null*, *u* è una radice.
- *visitato* è un array di boolean che ci dice quando un nodo è stato visitato, indicizzato anch'esso sui nodi del grafo.
- *color* è un array che mantiene il colore, ricordando che: *white* = non ancora trovato, *grey* = trovato ma non ancora “concluso”, *black* = totalmente processato.

Visita generica iterativa, in profondità o ampiezza (DFS e BFS)

A seconda della struttura dati (*Stack* o coda *FIFO*) abbiamo rispettivamente una visita in profondità con albero *DFS* oppure una visita in ampiezza con albero *BFS*.

```
vistaAll(Grafo G){
    /* Inizializziamo le strutture */
    int[] padri = new int[numNodiGrafo];
    * creazione dell'array color (con enum white, grey, black)
    * inizializza color a white
    * creazione della struttura d'appoggio (F)

    foreach(Nodo u in G) { //per i grafi non connessi
        if(color[u] == white)
            visit(u, padri, color, F);
    }
}

visit(Nodo S, int[] padri, color, F) {
    * inserisci S in F
    padri[S] = null;
    color[S] = grey;

    while(!isEmpty(F)) {
        * estrai Nodo u da F //POP – lettura da coda
        visita(u);
        color[u] = black;

        foreach(Nodo v adiacente a u) {
            if(color[S] == white && !appartiene(v,F)) {
                * inserisci Nodo v in F; //PUSH – messa in coda
                color[v] = grey;
                padre[v] = u;
            } //fine if
        } //fine foreach
    } //fine while
}
```

Complessità: $O(m+n)$

Visita in profondità ricorsiva (DFS)

```
vistaAll(Grafo G){  
    /* Inizializziamo le strutture */  
    int[] padri = new int[numNodiGrafo];  
    boolean visitato = new boolean[numNodiGrafo];  
  
    foreach(Nodo u in G) { //per i grafi non connessi  
        if(!visitato[u])  
            visitaRicorsivaDFS(u, padri, visitato);  
    }  
}  
  
visitaRicorsivaDFS(Nodo u, int[] padri, boolean[] visitato) {  
    inizio_visita(u);  
    visitato[u] = true;  
  
    foreach(Nodo v adiacente a u) {  
        if(!visitato[v]){  
            padri[v] = u;  
            visitaRicorsivaDFS(v, padri); //chiamata ricorsiva  
        }  
    }  
    termine_visita(u);  
}
```

Complessità:

Calcolo del cammino minimo (algoritmo di Dijkstra)*GRAFI PESATI ORIENTATI.*

Si definisce cammino minimo il path che riesce a percorrere tutti i nodi con minor costo possibile. Il costo di un path è la somma dei costi dei singoli archi che lo compongono.

Notazioni:

- $dist$ è l'array che mantiene le distanze, $dist[i]$ è la distanza (peso!) del nodo i esimo dalla sorgente, ovvero il valore del path fino a i , ovvero la somma dei pesi degli archi che costituiscono il path dalla sorgente fino al nodo i . La $dist[u]$ dove u non è ancora stato trovato vale infinito.
- Q è una coda con priorità (la priorità degli elementi è il peso del loro arco).
- $c_{u,v}$ è il costo dell'arco che va da u a v .

L'algoritmo è una rivisitazione dell'algoritmo di percorrenza in ampiezza con tre colori. (`camminoMinimoBase()`).

```
camminoMinimoBase(Grafo G, Sorgente S) {
    foreach(Nodo u appartenente a G) color[u] = white;
    Q = new Queue(); //creazione coda vuota
    padri[S] = null;
    dist[S] = 0; //è la radice!
    * inserisci S in Q
    color[S] = grey;
    /* I.C. per ogni Nodo u per cui color[u] = black, dist[u] è la lunghezza
    del cammino fino a u dalla sorgente.
    Per ogni Nodo v per cui color[v] = grey, dist[v] è la lunghezza del
    cammino minimo fino ad ora trovato (ovvero la path è composta da neri) più
    un possibile candidato (v stesso). */
    while(!isEmpty(Q)) {
        u = estraiMinimo(Q);
        color[u] = black;
        foreach(Nodo v adiacente a u) {
            if(color[v] == white || (dist[u] +  $c_{u,v}$ ) < dist[v]) {
                dist[v] = dist[u] +  $c_{u,v}$ ;
                padri[v] = u;
                if(color[v] == white) {
                    * inserisci V in Q (con priorità dist[v])
                    * color[v] = grey
                } else {
                    * cambia priorità di v in dist[v] in Q
                }
            } //fine if
        } //fine foreach
    } //fine while
}
```

Le operazioni di cambiamento di priorità servono per mantenere l'invariante che $dist[u]$ sia la distanza dal nodo fino a u , e quindi la sua priorità per cui sceglierlo (al giro dopo) deve variare.

Il confronto $(dist[u] + c_{u,v}) < dist[v]$ controlla invece che non vi siano nodi in coda che hanno già priorità migliore rispetto a quella dell' u appena trovato.

Consideriamo poi però le ottimizzazioni tali per cui $\text{visitato}[u] = \text{false}$ equivale a $\text{color}[u] = \text{white/grey}$, mentre $\text{visitato}[u] = \text{true}$ equivale a $\text{color}[u] = \text{black}$.

Ovvero introduciamo subito in coda tutti i nodi del grafo (così evitiamo il doppio if nel while), i quali verranno già messi in maniera ordinata. Questo ci fa risparmiare un colore.

Ci rendiamo però conto, a questo punto, che il confronto $(\text{dist}[u] + c_{u,v}) \leq \text{dist}[v]$ è più che sufficiente per verificare se un nodo sia già stato trovato o meno, infatti un nodo n con $\text{visitato}[n] = \text{false}$ avrà $\text{dist}[n] = \infty$ che sarà necessariamente maggiore di qualsiasi altra distanza, pertanto entreremo nell'if comunque. In questo modo eliminiamo i colori ed otteniamo un codice ottimizzato così (`camminoMinimoOptimized()`):

```
camminoMinimoOptimized(Grafo G, Sorgente S) {
    Q = new Queue(); //creazione coda vuota
    foreach(Nodo u appartenente a G) dist[u] = ∞;
    padri[S] = null;
    dist[S] = 0; //è la radice!
    foreach(Nodo u appartenente a G) * inserisci u in Q con priorità dist[u]
    /* I.C. per ogni Nodo u per cui color[u] = black, dist[u] è la lunghezza
    del cammino fino a u dalla sorgente.
    Per ogni Nodo v per cui color[v] = grey, dist[v] è la lunghezza del
    cammino minimo fino ad ora trovato (ovvero la path è composta da neri) più
    un possibile candidato (v stesso). */
    while(!isEmpty(Q)) {
        u = estraiMinimo(Q);
        foreach(Nodo v adiacente a u) {
            if((dist[u] + cu,v) <= dist[v]) {
                dist[v] = dist[u] + cu,v;
                padri[v] = u;
                * cambia priorità di v in dist[v] in Q
            }
        } //fine if
    } //fine foreach
} //fine while
}
```

Complessità:

- Con heap: $O((n+m) * \log(n))$
- Con lista non ordinata: $O(n^2)$

Costruzione dell'albero minimo ricoprente (algoritmo di Prim)

L'albero minimo ricoprente rappresenta il percorso effettuato sul grafo a partire da un nodo per raggiungere con costo minimo tutti gli altri nodi (ovvero toccati tutti).

Prendiamo la versione a due colori dell'algoritmo di Dijkstra (ovvero con l'array boolean visitato e tutti i nodi inseriti fin da subito) e non facciamo altro che togliere $dist[u]$ nei confronti/assegnamenti: infatti ora non ci interessa conteggiare tutto il percorso precedente bensì solo il nodo stesso ed il successivo ($c_{u,v}$ quindi).

```
camminoMinimoRicoprente(Grafo G, Sorgente S) {
    Q = new Queue(); //creazione coda vuota
    foreach(Nodo u appartenente a G) dist[u] = ∞;
    foreach(Nodo u appartenente a G) visitato[u] = false;
    padri[S] = null;
    dist[S] = 0; //è la radice!
    foreach(Nodo u appartenente a G) * inserisci u in Q con priorità dist[u]
    /* Per ogni Nodo x visitato, (padri[x], x) è un sottoalbero dell'albero di
    copertura minimo.
    Per ogni Nodo y non visitato, se dist[y] != infinito allora (padri[y],y) è
    un arco di peso minimo che collega y ad una serie di nodi neri.
    dist[u] è il peso del singolo arco, non del path! */
    while(!isEmpty(Q)) {
        u = estraiMinimo(Q);
        visitato[u] = true;
        foreach(Nodo v adiacente a u) {
            if(visitato[v] == false && ( $c_{u,v} < dist[v]$ )) {
                dist[v] =  $c_{u,v}$ ;
                padri[v] = u;
                * cambia priorità di v in dist[v] in Q
            }
        } //fine if
    } //fine foreach
} //fine while
}
```

Dimostrazione: regola del taglio.

Complessità: identico all'algoritmo di Dijkstra, ovvero $O((n+m) * \log(n))$.

Costruzione dell'albero minimo ricoprente (algoritmo di Kruskal)

L'albero minimo ricoprente rappresenta il percorso effettuato sul grafo a partire da un nodo per raggiungere con costo minimo tutti gli altri nodi (ovvero toccati tutti).

L'algoritmo di Kruskal semplicemente ordina in ordine decrescente di priorità (ovvero di peso) i nodi in una sequenza A. Una volta fatto questo, si scorre la lista e mano a mano vengono inseriti gli archi nel solito array padri. Per controllare che non si formino ciclicità, è sufficiente utilizzare una Union Find, laddove ogni volta che un nodo viene inserito nell'albero ricoprente, viene inserito nello stesso insieme di tutti i suoi nodi "sopra". Il numero di insieme sarà quindi pari al numero di foreste generate dall'algoritmo.

```
Kruskal(Grafo G) {
    * ordina gli archi in una sequenza A (non decrescente)
    * crea una Union Find con tutti i nodi di G come singoletti
    int[] padri = new int[numNodiGrafo];
    for(int i = 0; i < numNodiGrafo; i++) {
        * estrai da A l'arco (u,v) più leggero (il primo!);
        if(find(u) == find(v)) { //restituisce i rapp. controllo cicli
            padri[v] = u;
            union(u,v);
        }
    } //fine for
}
```

Sono possibili diverse ottimizzazioni. Innanzi tutto utilizzando la union che returna un true/false a seconda che i due nodi appartengano o meno allo stesso gruppo possiamo ridurre le operazioni dell'if. Inoltre, avendo n vertici ci bastano (n-1) archi per creare un albero ricoprente, gli altri verranno scartati automaticamente. Inseriamo pertanto un counter che va fino a numNodiGrafo - 1. Abbiamo perciò:

```
KruskalOptimized(Grafo G) {
    * ordina gli archi in una sequenza A (non decrescente)
    * crea una Union Find con tutti i nodi di G come singoletti
    int[] padri = new int[numNodiGrafo];
    int counter = 1;
    while(counter <= (numNodiGrafo-1)) {
        * estrai da A l'arco (u,v) più leggero (il primo!);
        if(union(u,v)) { //restituisce i rapp. controllo cicli
            padri[v] = u;
            counter++;
        }
    } //fine while
}
```

Complessità: $O(m * \log(m))$

Costruzione dell'albero topologico (grafi DAG - aciclici)

L'albero topologico di un grafo è un albero in cui i nodi sono collegati in questo modo: l'arco (u,v) esiste solo se u precede v nel grafo. Da un solo grafo sono ottenibili diversi alberi topologici: è possibile mettere i nodi in ordine differente purché sia rispettata la regola sopracitata.

Definiamo come **sorgenti** i nodi che non hanno archi entranti e **pozzi** quei nodi che non hanno archi uscenti.

Potremmo, dato un nodo sorgente, passare a tutti i suoi adiacenti, eliminare la connessione precedente, ed una volta trovato un nuovo sorgente procedere così. Ma trovare sorgenti è dispendioso. Assegniamo invece un grado entrante ad ogni vertice (grado entrante = numero di vertici incidenti sul nodo). Grado = 0 equivale ad avere un nodo sorgente.

- S sarà la lista di elementi da controllare (ovvero la coda FIFO, per esempio).
- ord è la lista su cui troveremo l'ordinamento topologico
- $grado[u]$ è la cella che contiene il grado del nodo u .

Vediamo una prima versione dell'algoritmo:

```
ordinamentoTopologico1(Grafo G) {
    S = new List();
    ord = new List();
    int[] grado = new int[numNodiGrafo];
    foreach(Nodo u in G) * assegna grado[u]; //tempo O(m)
    for(int i = 0; i < numNodiGrafo, i++)
        if(grado[i] == 0) * inserisci i in S; //tempo O(n), lista sorgenti
    /* S è l'insieme dei sorgenti del sottografo G'|V-ord| [V è l'insieme dei
    vertici di G].
    ord è un ordinamento topologico dei vertici G'|ord| */
    while(!isEmpty(S)) {
        * estrai u da S
        * inserisci u in coda ad ord
        foreach(Nodo v adiacente a u) {
            grado[v]--;
            if(grado[v] == 0) * inserisci v in S
        }
    } //fine while
    /* ord = V */
}
```

Complessità:

È oltremodo possibile, durante una visita in profondità (albero DFS) creare l'albero topologico. È sufficiente controllare il tempo di fine visita (tfs()). L'ordine dei tempi di fine visita ci da un ordine topologico invertito. Per comprendere questo fatto, basti pensare al tempo trascorso dai metodi ricorsivi sullo stack. Avremo bisogno dell'array `visitato` per evitare cicli.

```
ordinamentoTopologicoDFS(Grafo G) {  
    boolean visitato = new boolean[numNodiGrafo]; //inizializzato a false  
    ord = new List();  
    foreach(Nodo u in G)  
        if(!visitato[u]) ordinamentoTopologicoDFSRic(G, u, ord);  
}  
  
ordinamentoTopologicoDFS(Grafo G, Nodo u, Lista ord) {  
    visitato[u] = true;  
    foreach(Nodo v adiacente a u) {  
        if(!visitato[v]) {  
            ordinamentoTopologicoDFSRic(G, v, ord);  
        }  
    }  
    * inserisci u in testa ad ord  
}
```

Come è chiaro, l'inserimento viene fatto in testa poiché `tfs(u)` crea un ordine inverso rispetto a quello topologico.

Trovare le componenti fortemente connesse

- Visita in profondità del grafo G inserendo i nodi nella sequenza S in ordine crescente di tempo di fine visita,
- generare il grafo G^T trasposto di G ,
- creare una sequenza vuota `OrdineTopologicoDiCFC`
- per ogni elemento u di S dall'ultimo fino al primo:
 - crea un nuovo cfc vuoto;
 - visita in profondità in G^T tutti i nodi raggiungibili da u e non ancora visitati, aggiungendoli via via a C
 - aggiungi C al fondo della sequenza `OrdineTopologicoDiCFC`

Tutto ha senso, poiché generando l'ordine crescente di tempi di fine visita abbiamo un ordine del grafo "dal più profondo" fino ad arrivare ai sorgenti (a fondo lista).

Generando il grafo trasposto possiamo prendere, partendo dai sorgenti (fondo di S) di G (che sono i pozzi di G^T), e possiamo ricreare le componenti fortemente connesse.

Distinzione degli archi di un grafo (grafo orientato)

Tramite la nozione di visita DFS possiamo dividere gli archi in quattro categorie:
Prendiamo l'algoritmo di visita DFS:

```

vistaAll(Grafo G){
    /* Inizializziamo le strutture */
    int[] padri = new int[numNodiGrafo];
    boolean visitato = new boolean[numNodiGrafo];

    foreach(Nodo u in G) { //per i grafi non connessi
        if(!visitato[u])
            visitaRicorsivaDFS(u, padri, visitato);
    }
}

visitaRicorsivaDFS(Nodo u, int[] padri, boolean[] visitato) {
    inizio_visita(u);
    visitato[u] = true;

    foreach(Nodo v adiacente a u) {
        if(!visitato[v]){
            padri[v] = u;
            visitaRicorsivaDFS(v, padri); //chiamata ricorsiva
        }
    }
    termine_visita(u);
}

```

tramutiamo il codice in una visita a tre colori con gli array padri, color, d (tempo di inizio visita), f (tempo di fine visita). Consideriamo inoltre l'intero "time" per tenere traccia degli istanti trascorsi.

```

visitaRicorsivaDFS(Nodo u, int[] padri, ...) {
    color[u] = grey;
    d[u] = ++time;

    foreach(Nodo v adiacente a u) {
        if(color[v] == white){
            padri[v] = u;
            visitaRicorsivaDFS(v, padri, ...); //chiamata ricorsiva
        }
    }
    color[u] = black;
    f[u] = ++time;
}

```

Modificando debitamente gli if possiamo definire 4 categorie:

- 1) Arco appartenente all'albero, (lo trovo bianco, lo inserisco in T)
- 2) Arco all'indietro (*backward*), (lo trovo grigio)
- 3) Arco in avanti (*forward*), (lo trovo nero e v è discendente di u, ovvero $d[u] < d[v]$)
- 4) Arco di attraversamento (*cross*), (lo trovo nero e v non discende da u, ovvero $d[v] < d[u]$) - i due nodi non sono sullo stesso percorso.

Distinzione degli archi di un grafo (grafo non orientato)

Possiamo definire anche qui due categorie.

- 1) Arco appartenente all'albero (lo trovo non_visitato, lo inserisco in T)
- 2) Arco all'indietro (lo trovo visitato, ovvero si ricollega da qualche parte)